



**INSTITUTO LATINO-AMERICANO DE CIENCIAS
DA VIDA E DA NATUREZA (ILACVN)**

**PROGRAMA DE POS-GRADUACAO EM FISICA
APLICADA**

Circuitos Quânticos

Dylan Rafael Sánchez Salinas

Foz do Iguaçu

2026

SUMÁRIO

CIRCUITO DE BELL.....	3
ALGORITMO DE DEUTSCH-JOZSA	6
ALGORITMO DE BERNSTEIN-VAZIRANI.....	13
CIRCUITO DE SIMON	17

LISTA DE FIGURAS

Figura 1 – Circuito quântico para a geração de um estado de Bell.

Figura 2 – Circuito quântico do algoritmo de Deutsch-Jozsa.

Figura 3 – Histograma dos resultados obtidos para o algoritmo de Deutsch-Jozsa.

Figura 4 – Circuito quântico do algoritmo de Bernstein-Vazirani para a cadeia secreta $a=11001$, $a=11001$, $a=11001$.

Figura 5 – Histograma dos resultados obtidos para o algoritmo de Bernstein-Vazirani com cadeia secreta $a=11001$, $a=11001$, $a=11001$.

Figura 6 – Circuito quântico do algoritmo de Simon para a chave secreta $s=010$, $s=010$, $s=010$.

Figura 7 – Histograma dos resultados obtidos para o algoritmo de Simon com chave secreta $s=010$, $s=010$, $s=010$.

Circuito de Bell:

É importante compreender alguns circuitos fundamentais que ilustram os principais fenômenos da computação quântica. Entre esses fenômenos, destacam-se superposição e emaranhamento, que não possuem uma analogia direta na computação clássica.

Um dos exemplos mais simples e importante é o circuito utilizado para gerar um estado de Bell. Esse circuito (Figura 1) mostra como, a partir de um estado inicial separável, é possível obter um estado emaranhado por meio da aplicação de portas quânticas. Para isso, utiliza-se inicialmente uma porta de Hadamard, responsável por criar uma superposição, seguida de uma porta *CNOT*, responsável por correlacionar os dois qubits.

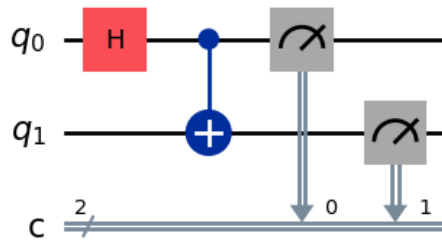


Figura 1: Circuito quântico de Bell

Considera-se o sistema inicialmente no estado:

$$|00\rangle \quad (1)$$

A primeira operação aplicada é a porta de Hadamard, no primeiro qubit. Sabemos que a ação da porta sobre o estado é dada por:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad (2)$$

Como a porta de Hadamard atua apenas no primeiro qubit, enquanto o segundo permanece no estado $|0\rangle$, o estado total do sistema passa a ser:

$$H|00\rangle \rightarrow \left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \right) \otimes |0\rangle \quad (3)$$

Que também pode ser escrito como:

$$\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \right) \otimes |0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \quad (4)$$

Então:

$$H|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \quad (5)$$

Em seguida, aplica-se a porta *CNOT*, utilizando o primeiro qubit de controle e o segundo como qubit alvo. A porta *CNOT* atua da seguinte maneira, se o qubit de controle estiver no estado $|0\rangle$, o qubit alvo permanece inalterado, se o qubit de controle estiver no estado $|1\rangle$, o qubit alvo será invertido.

Então:

$$CNOT|00\rangle = |00\rangle \quad (6)$$

$$CNOT|10\rangle = |11\rangle \quad (7)$$

Aplicando a operação *CNOT*, no estado de superposição (equação 5)

$$= \frac{1}{\sqrt{2}} CNOT(|00\rangle + |10\rangle) \quad (8)$$

Usando as equações (6) e (7) em (8), o estado final do sistema é dado por:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (9)$$

Esse estado é conhecido como um dos estados de Bell, ele representa um estado emaranhado, pois não pode ser escrito como estados individuais independentes.

Como os coeficientes associados a esses estados possuem o mesmo modulo, as probabilidades de medição são:

$$P(00) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} \quad (10)$$

$$P(11) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} \quad (11)$$

Portanto, ao executar esse circuito várias vezes, espera-se observar aproximadamente metade das medições no estado $|00\rangle$ e metade no estado $|11\rangle$. Esse resultado evidencia a correlação entre os dois

qubits: sempre que o primeiro é medido como 0, o segundo também é medido como 0; análogamente para o qubit que é medido como 1, o segundo também será medido como 1.

Implementação do Circuito de Bell

O código abaixo cria um circuito com dois qubits e dois bits clássicos, aplica uma porta de Hadamard no primeiro qubit, seguida de uma porta CNOT, e por fim realiza a medição dos dois qubits.

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

# Criar circuito com 2 qubits e 2 bits clássicos
qc = QuantumCircuit(2, 2)

# Passo 1: Aplicar Hadamard no qubit 0
qc.h(0)

# Passo 2: Aplicar CNOT com controle no qubit 0 e alvo no qubit 1
qc.cx(0, 1)

# Medir ambos os qubits
qc.measure([0, 1], [0, 1])

# Desenhar o circuito
qc.draw("mpl")
```

Para obter o resultado das medições, utiliza-se o simulador *AerSimulator*:

```
# Criar o simulador
simulator = AerSimulator()

# Executar o circuito no simulador
job = simulator.run(qc, shots=1024)

# Obter o resultado da simulação
result = job.result()

# Obter as contagens das medições
counts = result.get_counts()

# Mostrar as contagens
print(counts)

# Plotar o histograma
fig = plot_histogram(counts)
display(fig)
```

Após a execução do circuito se obtém:

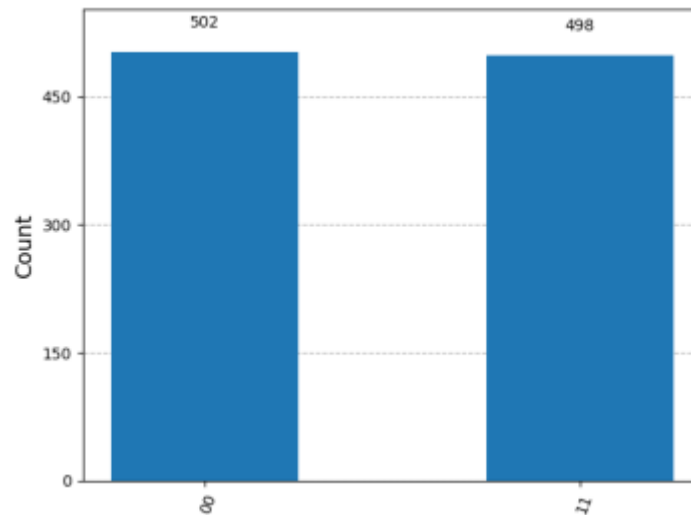


Figura 2: Histograma dos resultados das medições dos estados de Bell.

Algoritmo de Deutsch-Jozsa:

O algoritmo de Deutsch é um dos primeiros exemplos que mostram como a computação quântica pode resolver determinados problemas de forma mais eficiente que a computação clássica. Embora seja uns algoritmos simples, ele introduz uma ideia essencial: utilizar superposição, oráculos quânticos e interferência para extrair uma propriedade global de uma função com apenas uma consulta de oráculo.

O problema consiste em determinar se uma função booleana

$$f: \{0,1\} \rightarrow \{0,1\} \quad (13)$$

É constante ou balanceada. Uma função é chamada de constante quando produz o mesmo resultado para todas as entradas, isto é:

$$f(0) = f(1) \quad (14)$$

Por outro lado, uma função é chamada de balanceada quando produz resultados diferentes para as duas entradas possíveis:

$$f(0) \neq f(1) \quad (15)$$

Classicamente, para determinar com certeza se a função é constante ou balanceada, seria necessário avaliar a função nas duas entradas, 0 e 1. O algoritmo de Deutsch, no entanto, permite obter essa informação com apenas uma consulta ao oráculo quântico U_f .

O circuito utilizado é representado por meio da figura 3:

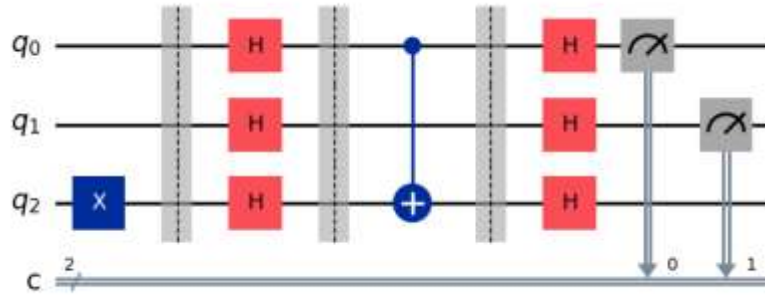


Figura 3: Circuito quântico do algoritmo de Deutsch-Jozsa.

Considerando o circuito apresentado na figura 3, formado por três qubits. Os qubits q_0 e q_1 representa o registrador de entrada, enquanto o qubit q_2 atua como qubit auxiliar. O sistema inicia no estado:

$$|000\rangle \quad (16)$$

Em seguida, aplica-se uma porta X no qubit auxiliar q_2 , transformando-o $|0\rangle \rightarrow |1\rangle$. Assim, o estado se torna:

$$|000\rangle \rightarrow |001\rangle \quad (17)$$

Portanto, antes da aplicação das portas de Hadamard, temos:

$$|\psi_0\rangle = |0\rangle|0\rangle|1\rangle \quad (18)$$

Sabemos que:

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (19)$$

Agora aplicamos a porta de Hadamard nos 3 qubits:

$$|\psi_1\rangle = H|0\rangle \otimes H|0\rangle \otimes H|1\rangle \quad (20)$$

Aplicamos as equações (2) e (19) em (20):

$$|\psi_1\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (21)$$

$$|\psi_1\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (22)$$

Onde:

$$|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (23)$$

Logo:

$$|\psi_1\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) \otimes |-\rangle \quad (24)$$

Aplicação do oráculo:

Na figura, o oráculo é representado por uma porta *CNOT* com controle no qubit q_0 e alvo no qubit auxiliar q_2 . Isso significa que o qubit auxiliar é invertido quando $q_0 = |1\rangle$.

Esse oráculo representa a função:

$$f(x_0, x_1) = x_0 \quad (25)$$

Portanto, temos uma função balanceada, pois metade das entradas produz $f = 0$ e metade produz $f = 1$:

$$f(0,0) = 0$$

$$f(0,1) = 0$$

$$f(1,0) = 1$$

$$f(1,1) = 1$$

No algoritmo de Deutsch-Jozsa, o oráculo atua como:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle \quad (26)$$

A expressão acima quer dizer o seguinte:

O oráculo recebe dois registros. O primeiro é $|x\rangle$, que representa a entrada da função. O segundo é $|y\rangle$, que é o qubit auxiliar.

O ponto importante é: o oráculo não muda o valor de x . Ele mantém a entrada igual. O que ele modifica é o qubit auxiliar y , dependendo do valor da função $f(x)$.

O símbolo $\oplus$ significa a soma módulo 2:

$$0 \oplus 0 = 0,$$

$$0 \oplus 1 = 1,$$

$$1 \oplus 0 = 1,$$

$$1 \oplus 1 = 0.$$

Então, se $f(x) = 0$, temos:

$$y \oplus f(x) = y \oplus 0 = y \quad (27)$$

Se $f(x) = 1$

$$y \oplus f(x) = y \oplus 1 \quad (28)$$

Nesse caso, o qubit auxiliar é invertido. Se ele era $|0\rangle$, se torna $|1\rangle$. Se ele era $|1\rangle$, se torna $|0\rangle$.

Então de forma simples:

$$f(x) = 0 \rightarrow \text{Auxiliar fica igual.}$$

$$f(x) = 1 \rightarrow \text{Auxiliar é invertido}$$

No Deutsch-Jozsa, o qubit auxiliar não está simplesmente em $|0\rangle$ ou em $|1\rangle$. Ele foi preparado no estado $|-\rangle$ (equação 23).

Esse estado é especial porque, quando aplicamos uma porta X nele, acontece isto:

$$X|-\rangle = \frac{1}{\sqrt{2}}X(|0\rangle - |1\rangle) = \frac{1}{\sqrt{2}}(|1\rangle - |0\rangle) = -\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = -|-\rangle \quad (29)$$

Ou seja, a porta X não transforma $|-\rangle$ em um estado completamente diferente. Ela apenas coloca um sinal negativo na frente.

Se $f(x) = 0$ se mantem o signo, se $f(x) = 1$, se agrega um sinal negativo. Esses dois casos podem ser escritos como:

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle \quad (30)$$

Aplicando o oráculo na equação (24):

$$|\psi_2\rangle = \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle) \otimes |-\rangle \quad (31)$$

Assim, os estados com $x_0 = 0$ mantêm sinal positivo, enquanto os estados com $x_0 = 1$ recebem o sinal negativo.

Podemos reorganizar colocando os termos $|0\rangle$ e $|1\rangle$ em evidencia:

$$|\psi_2\rangle = \frac{1}{2}(|0\rangle(|0\rangle + |1\rangle) - |1\rangle(|0\rangle + |1\rangle)) \otimes |-\rangle \quad (32)$$

Logo:

$$|\psi_2\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |-\rangle \quad (33)$$

Sabemos que:

$$\frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle \quad (34)$$

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle \quad (35)$$

Então:

$$|\psi_2\rangle = |-\rangle \otimes |+\rangle \otimes |-\rangle \quad (36)$$

Agora aplicamos novamente a portas de Hadamard nos qubits :

$$H|-\rangle = \frac{1}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) - \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right) = \frac{1}{2}((|1\rangle) + (|1\rangle)) = |1\rangle \quad (37)$$

$$H|+\rangle = \frac{1}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right) = \frac{1}{2}((|0\rangle) + (|0\rangle)) = |0\rangle \quad (38)$$

Aplicando as equações (37) e (38) em (36):

$$|\psi_f\rangle = |1\rangle \otimes |0\rangle \otimes |-\rangle \quad (39)$$

Ou também:

$$|\psi_f\rangle = |10\rangle \otimes |-\rangle \quad (40)$$

Ao medir os qubits de entrada, esperamos obter $|10\rangle$.

No algoritmo de Deutsch-Jozsa, a regra é:

$|00\rangle \rightarrow \text{Função constante}$

$\text{Qualquer estado diferente de } |00\rangle \rightarrow \text{função balanceada}$

No nosso caso como $|10\rangle \neq |00\rangle$, temos que a função implementada pelo oráculo é balanceada.

Implementação do Algoritmo de Deutsch-Jozsa:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display
import matplotlib.pyplot as plt

# Creamos el circuito con 3 qubits y 2 bits clásicos
qc_3 = QuantumCircuit(3, 2)
# Preparamos el qubit auxiliar en |1>
qc_3.x(2)
qc_3.barrier()
# Aplicamos Hadamard en los tres qubits
qc_3.h(0)
qc_3.h(1)
qc_3.h(2)
qc_3.barrier()
# Oráculo Uf
# CNOT con control en q0 y alvo en q2
# Esto representa f(x0,x1) = x0
qc_3.cx(0, 2)

qc_3.barrier()
# Aplicamos Hadamard nuevamente solo en los qubits de entrada
```

```

qc_3.h(0)
qc_3.h(1)
qc_3.barrier()
# Medimos los qubits de entrada
qc_3.measure(0, 0)
qc_3.measure(1, 1)
# Simulación
simulator = AerSimulator()
compiled = transpile(qc_3, simulator)
result = simulator.run(compiled, shots=1024).result()
counts = result.get_counts()
print("Counts:", counts)
hist = plot_histogram(counts)
display(hist)

```

Após a execução do circuito se obtém:

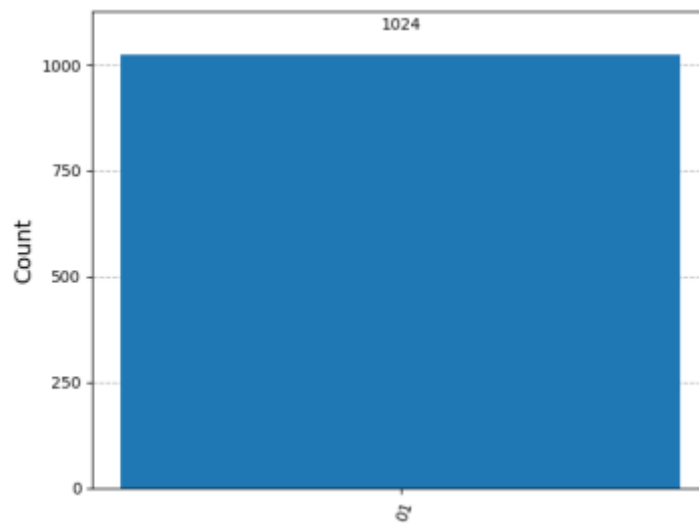


Figura 4: Histograma do algoritmo de Deutsch-Jozsa.

(Lembrando que em Qiskit os resultados do histograma se mantem na ordem dos bits clássicos)

Algoritmo de Berstein- Vazirani

O algoritmo de Berstein-Vazirani é um exemplo importante de algoritmo quântico baseado em superposição, oráculo quântico e interferência. Ele pode ser entendido como uma continuação natural do algoritmo de Deutsch-Jozsa, mas como um objetivo diferente. Enquanto o algoritmo Bernstein-Vazirani determina uma cadeia binária secreta escondida dentro de uma função.

O algoritmo de BV, considera uma função

$$f(x) = a \cdot x, \quad (\text{mod } 2) \quad (41)$$

Onde a é a cadeia secreta e x é a entrada da função. O produto $a \cdot x$ é calculado bit a bit, usando soma modulo 2. Assim, se:

$$a = a_0 a_1 a_2 \dots a_{n-1} \quad (42)$$

e

$$x = x_0 x_1 x_2 \dots x_{n-1} \quad (43)$$

Então:

$$a \cdot x = a_0 x_0 \oplus a_1 x_1 \oplus \dots a_{n-1} x_{n-1} \quad (44)$$

O símbolo $\oplus$ representa a soma modulo 2. Portanto, o resultado da função será sempre 0 ou 1.

No caso implementado no código, a cadeia “secreta” escolhida foi:

$$a = 11001 \quad (45)$$

Como essa cadeia possui cinco bits, o circuito utiliza cinco qubits de entrada e um qubit auxiliar. Assim, o sistema possui seis qubits no total.

O circuito implementado está representado na figura 5:

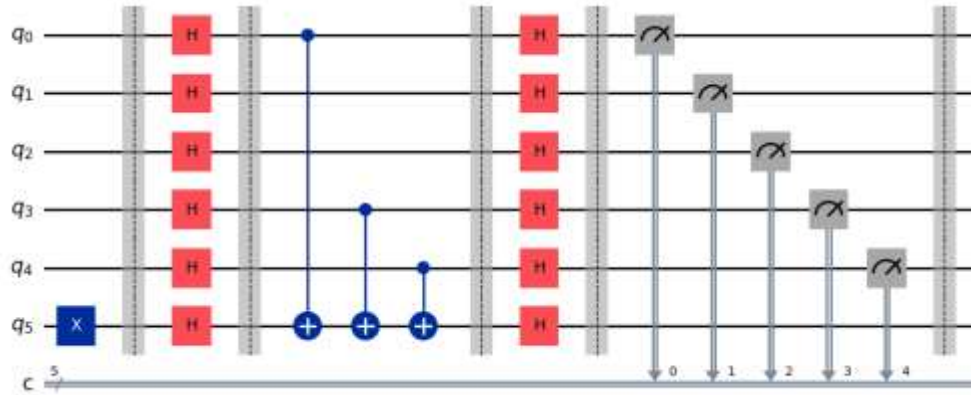


Figura 5: Circuito Quântico Bernstein-Vazirani

Qubits de entrada: $(q_0, q_1, q_2, q_3, q_4)$

Qubit auxiliar: (q_5)

O Circuito inicia no estado:

$$|00000\rangle|0\rangle \quad (46)$$

Em seguida, aplica-se uma porta X no qubit auxiliar:

$$|00000\rangle|1\rangle \quad (47)$$

Depois disso, são aplicadas portas de Hadamard em todos os qubits. Nos qubits de entrada, a Hadamard cria uma superposição de todas as possíveis entradas $x \in \{0,1\}^5$. Como existem cinco qubits de entrada, existem:

$$2^5 = 32 \quad (48)$$

Estados possíveis. Assim, o registrador de entrada passa a ser uma superposição uniforme de todos os estados:

$$\frac{1}{\sqrt{32}} \sum_{x \in \{0,1\}^5} |x\rangle \quad (49)$$

Lembrando que, o qubit auxiliar estava no estado $|1\rangle$, ao aplicar o Hadamard, transforma-se $|-\rangle$ (equação 23). Portanto, após a primeira camada de portas de Hadamard, o estado do sistema pode ser escrito como:

$$|\psi_1\rangle = \frac{1}{\sqrt{32}} \sum_{x \in \{0,1\}^5} |x\rangle \otimes |-\rangle \quad (50)$$

Essa expressão significa que o circuito está avaliando, em superposição, todas as possíveis entradas da função.

Aplicação do oráculo.

O oráculo do algoritmo de Bernstein-Vazirani é definido por:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle \quad (51)$$

Como no algoritmo de Deutsch-Jozsa, o qubit auxiliar foi preparado no estado $|-\rangle$, ocorre novamente, que a cada ação do oráculo sobre o qubit auxiliar aparece uma fase, ou seja:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle \quad (52)$$

Substituindo (41) em (52) temos:

$$U_f |x\rangle |-\rangle = (-1)^{a \cdot x} |x\rangle |-\rangle \quad (52)$$

Portanto, ao aplicar o oráculo ao estado $|\psi_1\rangle$, obtemos:

$$|\psi_2\rangle = \frac{1}{\sqrt{32}} \sum_{x \in \{0,1\}^5} (-1)^{a \cdot x} |x\rangle \otimes |-\rangle \quad (53)$$

Esse é o ponto central do algoritmo. O oráculo não revela diretamente a cadeia secreta a . Em vez disso, ele marca cada estado $|x\rangle$ com uma fase positiva ou negativa dependendo do valor de $a \cdot x$.

Note que o oráculo coloca as fases $(-1)^{a \cdot x}$. Após a aplicação do operador Hadamard (novamente) transforma esse padrão de fases na cadeia binária a . Em outras palavras, a informação que estava escondida nos sinais de superposição passa a aparecer diretamente como um estado mensurável

$$|a\rangle = |11001\rangle \quad (54)$$

Implementação do circuito quântico BV:

Nota: O oráculo aplica $CNOT$ s apenas nos qubits associados aos bits iguais a 1.

```

# Importamos as bibliotecas

from qiskit import QuantumCircuit

from qiskit_aer import AerSimulator

# Cadeia secreta

a = "11001"

n = len(a)

# Criar o circuito:

# n qubits de entrada + 1 qubit auxiliar

# n bits clássicos para a medição

qc = QuantumCircuit(n + 1, n)

# Preparamos o qubit auxiliar no estado  $|1\rangle$ 

qc.x(n)

qc.barrier()

# Aplicamos Hadamard em todos os qubits

qc.h(range(n + 1))

qc.barrier()

# Aplicamos o oráculo

# Para cada bit igual a 1 na cadeia secreta, aplicamos uma CNOT

for i, bit in enumerate(reversed(a)):

    if bit == "1":

        qc.cx(i, n)

qc.barrier()

# Aplicamos Hadamard novamente

qc.h(range(n + 1))

qc.barrier()

```



```

# Medição dos qubits de entrada

qc.measure(range(n), range(n))

qc.barrier()

# Executar no simulador

simulator = AerSimulator()

result = simulator.run(qc, shots=1024).result()

counts = result.get_counts()

print("Resultados:", counts)

```

Algoritmo de Simon

O algoritmo de Simon é um dos algoritmos quânticos fundamentais que mostram como a computação quântica pode explorar a superposição, oráculos e interferência para descobrir propriedades escondidas em uma função. Ele pode ser entendido como uma continuação natural dos algoritmos de Deutsch-Jozsa e Bernstein-Vazirani, mas com uma diferença importante: no algoritmo de Simon, a cadeia secreta não é obtida diretamente por uma única medição.

No algoritmo de Bernstein-Vazirani, o objetivo era encontrar diretamente uma cadeia secreta a , associada a função (equação 41)

Já no algoritmo de Simon, o objetivo é encontrar uma cadeia secreta s , chamada de período, tal que satisfaça a propriedade:

$$f(x) = f(x \oplus s) \quad (55)$$

Essa expressão significa que duas entradas diferentes, x e $x \oplus s$, produzem a mesma saída de função. Portanto, a função possui uma simetria escondida, determinada pela cadeia secreta s .

O símbolo $\oplus$ representa soma modulo 2 bits a bit. Por exemplo, se:

$$x = 101$$

E

$$s = 011$$

Então:

$$x \oplus s = 101 \oplus 011 = 110$$

Assim, o algoritmo de Simon busca descobrir qual é a cadeia s que relaciona pares de entradas que produzem a mesma saída.

O circuito de Simon começa preparando uma superposição de todas as entradas possíveis. Em seguida, o oráculo é aplicado. Esse oráculo codifica a função $f(x)$, de modo que entradas relacionadas por s produzem a mesma saída (equação 55)

Depois de aplicação do oráculo e da medição do segundo registrador, o primeiro registrador fica em uma superposição de dois estados relacionados:

$$|x\rangle + |x \oplus s\rangle \quad (56)$$

Em seguida, aplica-se portas de Hadamard ao primeiro registrador. A interferência produzida por essas portas faz com que os resultados possíveis de medição não sejam quaisquer estados, mas apenas aqueles que satisfazem a condição:

$$y \cdot s = 0 \pmod{2} \quad (57)$$

Essa é a equação central do algoritmo de Simon.

Diferentemente de Bernstein-Vazirani, o algoritmo de Simon não fornece diretamente s . Em vez disso, cada medição fornece um valor y que obedece a equação (57). Após várias medições, obtemos várias equações lineares. A partir delas, podemos resolver o sistema e encontrar a cadeia secreta s .

Para trabalhar com três bits, escrevemos:

$$s = s_0 s_1 s_2 \quad (58)$$

E:

$$y = y_0 y_1 y_2 \quad (59)$$

O produto interno módulo 2 é definido como:

$$y \cdot s = y_0 s_0 \oplus y_1 s_1 \oplus y_2 s_2 \quad (60)$$

Lembrando que a condição do algoritmo de Simon é:

$$y_0 s_0 \oplus y_1 s_1 \oplus y_2 s_2 = 0 \quad (61)$$

Ou seja, os valores medidos y devem ser ortogonais a cadeia secreta s , considerando aritmética módulo 2.

Chaves solicitadas:

$$s = 010, \quad 011, \quad 100, \quad 101, \quad 110 \quad (62)$$

Temos que encontrar, para cada uma delas os valores de y que satisfazem a equação (57).

Caso 1: $s = 010$

Temos:

$$s_0 = 0, \quad s_1 = 1, \quad s_2 = 0 \quad (63)$$

Então:

$$y \cdot s = y_0(0) \oplus y_1(1) \oplus y_2(0) \quad (64)$$

$$y \cdot s = y_1 \quad (65)$$

A condição de Simon é:

$$y_1 = 0 \quad (66)$$

(para satisfazer a equação 61)

Portanto, os valores possíveis de y são aqueles cujos segundo bit é zero (explicar melhor isto)

$$y \in \{000, \quad 001, \quad 100, \quad 101\} \quad (67)$$

Caso 2: $s = 011$

Então:

$$y \cdot s = y_0(0) \oplus y_1(1) \oplus y_2(1) \quad (68)$$

$$y \cdot s = y_1 \oplus y_2 \quad (69)$$

A condição de Simon é:

$$y_1 \oplus y_2 = 0 \quad (70)$$

Isso significa que y_1 e y_2 devem ser iguais. Portanto, os valores possíveis são:

$$y \in \{000, \quad 011, \quad 100, \quad 111\} \quad (71)$$

Caso 3: $s = 100$

Então:

$$y \cdot s = y_0(1) \oplus y_1(0) \oplus y_2(0) \quad (72)$$

$$y \cdot s = y_0 \quad (73)$$

A condição de Simon é:

$$y_0 = 0 \quad (74)$$

Portanto, os valores possíveis de y são aqueles cujos primer bit é zero

$$y \in \{000, \quad 001, \quad 010, \quad 011\} \quad (75)$$

Caso 4: $s = 101$

Então:

$$y \cdot s = y_0(1) \oplus y_1(0) \oplus y_2(1) \quad (76)$$

$$y \cdot s = y_0 \oplus y_2 \quad (77)$$

A condição de Simon é:

$$y_0 \oplus y_2 = 0 \quad (78)$$

Isso significa que y_0 e y_2 devem ser iguais. Portanto, os valores possíveis são:

$$y \in \{000, \quad 010, \quad 101, \quad 111\} \quad (79)$$

Caso 5: $s = 110$

Então:

$$y \cdot s = y_0(1) \oplus y_1(1) \oplus y_2(0) \quad (80)$$

$$y \cdot s = y_0 \oplus y_1 \quad (81)$$

A condição de Simon é:

$$y_0 \oplus y_1 = 0 \quad (82)$$

Isso significa que y_0 e y_1 devem ser iguais. Portanto, os valores possíveis são:

$$y \in \{000, \quad 001, \quad 110, \quad 111\} \quad (83)$$

A tabela 1: mostra que, para cada cadeia secreta s , existem apenas alguns resultados de y permitidos. Esses resultados satisfazem a condição da equação 57.

Tabela 1: Resumo

CADEIA SECRETA s	CONDIÇÃO $y \cdot s = 0$	VALORES POSSÍVEIS DE y
010	$y_1 = 0$	000, 001, 100, 101
011	$y_1 = y_2$	000, 011, 100, 111
100	$y_0 = 0$	000, 001, 010, 011
101	$y_0 = y_2$	000, 010, 101, 111
110	$y_0 = y_1$	000, 001, 110, 111

Portanto, ao executar o algoritmo de Simon varias vezes, os resultados não devem apresentar todos os estados possíveis de maneira arbitraria. Ele deve apresentar apenas os estados compatíveis com a cadeia secreta s . Assim o algoritmo de Simon transforma a periodicidade escondida da função em um conjunto de equações lineares. A partir dessas equações, podemos construir a cadeia secreta s .

Exemplo: Implementação para a chave $s = 010$.

Como mostrado na Figura 6, o circuito utiliza três qubits de entrada, q_0, q_1 e q_2 e três qubits de saída q_3, q_4 e q_5 . Inicialmente, aplicam-se portas Hadamard nos qubits de entrada, criando uma superposição de todas as entradas possíveis. Em seguida o oráculo é aplicado, codificando a periodicidade para a chave selecionada, neste caso $s = 010$. Por fim, novas portas de Hadamard são aplicadas aos qubits de entrada antes da medição.

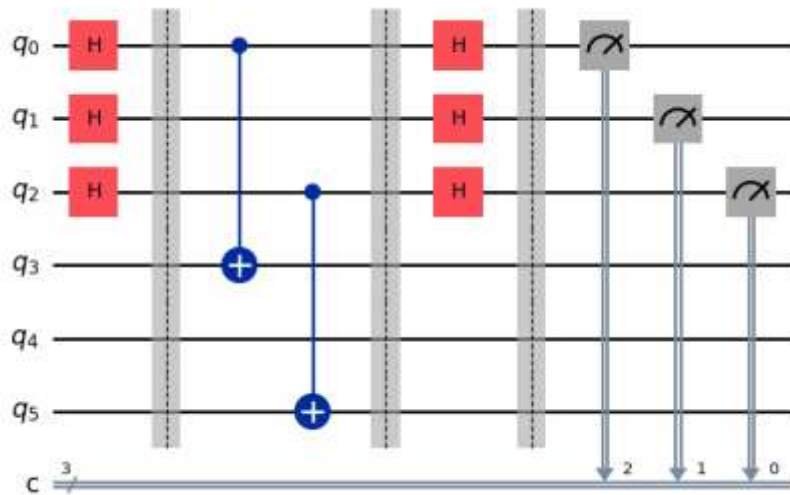


Figura 6: Algoritmo de Simon para $s = 010$.

Após a execução do circuito, obteve-se o histograma a seguir:

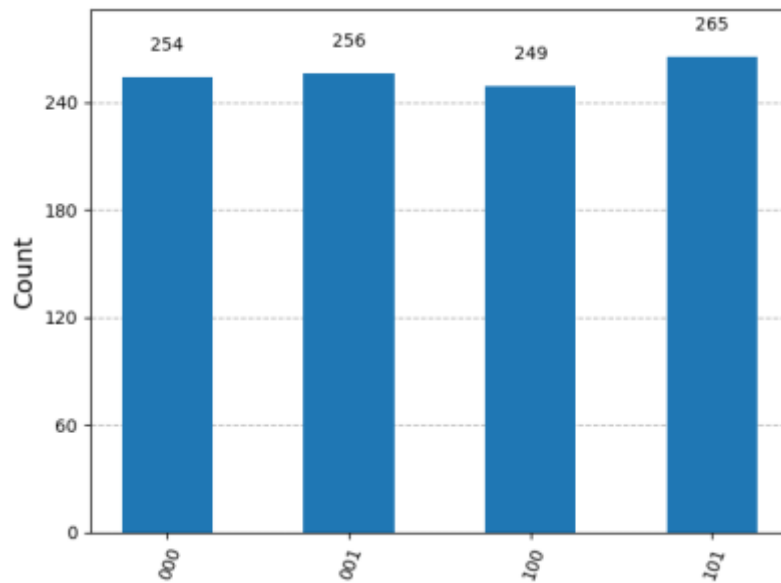


Figura 7: Histograma dos resultados do algoritmo de Simon para chave $s = 010$.

Código implementando para o algoritmo de Simon:

```
from qiskit import QuantumCircuit, transpile

from qiskit_aer import AerSimulator

from qiskit.visualization import plot_histogram

from IPython.display import display


# Escolha da chave secreta

# =====

s = "010"

# s = "011"

# s = "100"

# s = "101"

# s = "110"


n = len(s)


# Criamos o circuito

qc = QuantumCircuit(2*n, n)


# Hadamard nos qubits de entrada

qc.h(range(n))

qc.barrier()


# Oráculo de Simon

p = s.index("1")
```

```

for i, bit in enumerate(s):

    if i == p:

        continue

    qc.cx(i, n + i)

    if bit == "1":

        qc.cx(p, n + i)

qc.barrier()

# Hadamard novamente nos qubits de entrada
qc.h(range(n))

qc.barrier()

# Medição dos qubits de entrada
for i in range(n):

    qc.measure(i, n - 1 - i)

# Simulador

simulator = AerSimulator()

compiled = transpile(qc, simulator)

result = simulator.run(compiled, shots=1024).result()

counts = result.get_counts()

```



```
print("Chave secreta s =", s)

print("Resultados:", counts)

# Mostrar circuito em texto
print(qc.draw())

# Mostrar histograma
hist = plot_histogram(counts)

display(hist)
```